TicketsRUs: Online Movie Ticketing System

Software Requirements Specification

Version <3.0>

03/19/2026

Group 8
Sean Lumahan
Hinckclair Patino
Brady Santos

Prepared for
CS 250- Introduction to Software Systems
Instructor: Dr. Gus Hanna
Spring 2026

Revision History

Date	Description	Author	Comments
2/19/2026	Version <1.0>	Hinckclair Patino, Sean Lumahan, Brady Santos	First version of the Software Requirement Specification
3/05/2026	Version <2.0>	Hinckclair Patino, Sean Lumahan, Brady Santos	UML and software architecture diagrams, designs, descriptions, etc.
3/19/2026	Version <3.0>	Hinckclair Patino, Sean Lumahan, Brady Santos	Test cases added: cases, descriptions, results, etc.

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Dr. Gus Hanna	Instructor, CS 250	
	Sean Lumahan		
	Hinckclair Patino		
	Brady Santos		

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 1
1.4 References 1
1.5 Overview 1

2. General Description 2

2.1 Product Perspective 2
2.2 Product Functions 2
2.3 User Characteristics 2
2.4 General Constraints 2
2.5 Assumptions and Dependencies 2

3. Specific Requirements 2

3.1 External Interface Requirements 3
3.1.1 User Interfaces *3*
3.1.2 Hardware Interfaces *3*
3.1.3 Software Interfaces *3*
3.1.4 Communications Interfaces *3*
3.2 Functional Requirements 3
3.2.1 <Functional Requirement or Feature #1> *3*
3.2.2 <Functional Requirement or Feature #2> *3*
3.3 Use Cases 3
3.3.1 Use Case #1 ..... *3*
3.3.2 Use Case #2 ..... *3*
3.4 Classes / Objects 3
3.4.1 <Class / Object #1> *3*
3.4.2 <Class / Object #2> *3*
3.5 Non-Functional Requirements 4
3.5.1 Performance *4*
3.5.2 Reliability *4*
3.5.3 Availability *4*
3.5.4 Security *4*
3.5.5 Maintainability *4*
3.5.6 Portability *4*
3.6 Inverse Requirements 4
3.7 Design Constraints 4
3.8 Logical Database Requirements 4
3.9 Other Requirements 4

4. Analysis Models 4

4.1 Sequence Diagrams 5
4.3 Data Flow Diagrams (DFD) 5
4.2 State-Transition Diagrams (STD) 5

5. Change Management Process 5

A. Appendices 5

A.1 Appendix 1 5
A.2 Appendix 2 5

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for **TicketsRUs**, a digital movie ticketing platform designed to streamline the process of browsing and purchasing theater tickets online. This document is intended for developers, testers, project stakeholders, and course instructors. It serves as a formal reference to guide system design, implementation, and evaluation.

The goal of this SRS is to clearly define all functional and non-functional requirements before development begins to reduce ambiguity and improve implementation accuracy.

1.2 Scope

TicketsRUs is a web-based and mobile-accessible platform that allows users to browse movies, view showtimes, and purchase digital tickets for participating theaters. The system also supports private theater reservations and digital ticket validation.

TicketsRUs will:

- Allow users to create and manage accounts
- Display movie listings, showtimes, and ratings
- Enable advance ticket purchases
- Deliver digital tickets prior to showtime
- Support private theater reservations
- Apply discounts and reward incentives (i.e. promo codes)
- Limit ticket purchases to reduce scalping

TicketsRUs will not:

- Allow ticket reselling between users
- Replace existing in-theater scanning systems
- Sell physical food or merchandise

The primary objective of TicketsRUs is to provide a fast, secure, and accessible ticket purchasing experience while supporting theater operations and minimizing fraudulent resale activity.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SRS	Software Requirements Specification
NFC	Near Field Communication
QR Code	Quick Response Code
API	Application Programming Interface
Concurrent User	A user actively interacting with the system at the same time
Scalping	The resale of tickets for profit

1.4 References

1.5 Overview

The document is organized as follows:
Section 1: Provides an introduction and scope of the system.
Section 2: Describes the overall system context, users, and constraints.
Section 3: Discusses expected requirements of the system along with sample use cases.
Section 4: Outlines analysis models used in the system design.
Section 5: Describes the change management process.

2. General Description

2.1 Product Perspective

The Movie Ticketing System is a standalone platform designed to integrate with participating theaters. As the product is intended as an online movie ticketing store, there are multiple similar systems already in place, such as Fandango and AMC. However, our system is geared toward providing more information to users regarding movies, as well as providing them with numerous accessibility services that may not be available on all other online ticketing systems. The system does not replace existing theater infrastructure, but operates alongside it.

The platform will be designed in relation to:

- Theater ticket validation systems
- Third-party movie data APIs
- External payment processing services

2.2 Product Functions

- Allow users to create and manage personal accounts
- Display movies, showtimes, and ratings
- Enable ticket purchasing and seat selection
- Deliver digital tickets prior to showtime
- Enforce ticket purchase limits
- Support private theater reservations
- Apply discounts and reward points
- Notify users of purchase confirmations and ticket release

The system prioritizes speed, usability, and security to ensure a smooth transaction experience.

2.3 User Characteristics

Primary users are individuals aged 13 and older seeking to purchase movie tickets digitally. Users are expected to have basic familiarity with mobile applications or web platforms, including account login, navigation, and online payment processes.

Theater staff will also utilize the software to validate tickets using scanning devices, as well as administrators who manage showtimes and theater participation.

Overall, the system should be intuitive and accessible for users with varying levels of technical proficiency.

2.4 General Constraints

The software system is subject to the following general constraints:

- Internet connectivity is required for access and ticket purchases
- The system must comply with regional tax and pricing regulations
- The system must operate using existing theater scanning hardware.
- The platform must support modern web browsers and mobile devices (iOS and Android)
- The UI must remain simple, intuitive, and fast to minimize checkout time

2.5 Assumptions and Dependencies

The system assumes:

- We assume participating theaters agree to integrate their showtimes, seat availability, and ticket scanning systems with this platform
- We assume the system has access to third-party APIs to provide accurate movie ratings, reviews, and related metadata.
- The system relies on external payment processors to complete financial transactions securely without interruption
- It is assumed users will have access to compatible devices capable of displaying digital tickets (e.g., smartphones or modern web browsers).
- The availability of reliable internet or cellular data services are required for users to access the system

If any of these dependencies change, system requirements may require revision.

3. Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- Fast and responsive web-based user interface. Needs to work on all browsers to avoid issues later on.
- Support both desktop and mobile and can adjust based on different screen sizes
- Supports auto-currency detection and perhaps language detection? Can be manually changed by the user as well.
- Users should be able to view movies, showtimes, seat availability, pricing, and reviews before purchasing the tickets.
- Compatible with mobile wallets (ex. Google and Apple Pay)

3.1.2 Hardware Interfaces

- Supports QR/NFC ticket scanning using standard theater ticket scanner.
- Allow users to present tickets from smartphones and be able to print them out if needed.

3.1.3 Software Interfaces

- The system shall integrate with third-party movie data APIs. (Ex. Movie ratings and reviews)
- The system shall interface with external payment gateways including Apple Pay, Google Pay, PayPal, credit cards, and debit cards

- The system shall interface with a database storing user accounts, tickets, showtimes, and transactions

3.1.4 Communications Interfaces

- The system shall use HTTPS for all the client-server communication.
- The system shall detect user location via IP address for currency and regional pricing adjustments. (Might need to ask users to accept our policy first so we can legally apply this idea)
- The system shall notify users via email or SMS when their tickets are released. The system sends out reminders as well if the user prefers. (Can be an option, user can disable)

3.2 Functional Requirements

3.2.1 User Account Management

3.2.1.1 Introduction
The system shall allow users to create and manage personal accounts.

3.2.1.2 Inputs
- Name
- Email Address
- Phone number (Optional)
- Date of birth
- Payment types and info

3.2.1.3 Processing
- Store user information securely
- Track reward points earned from purchases

3.2.1.4 Outputs
- Confirmation of account creation
- User profile dashboard

3.2.1.5 Error Handling
- Duplicate accounts shall not be allowed
- Invalid email formats will be rejected

3.2.2 Ticket Purchasing

3.2.2.1 Introduction
The system shall allow users to purchase movie tickets online.

3.2.2.2 Inputs

- Selected movie
- Showtime
- Number of tickets
- Payment methods

3.2.2.3 Processing
- Enforce a maximum of 20 tickets per user per movie unless renting an entire theater.
- Prevent purchases after ticket cutoff times (10 minutes before showtime).
- Delay ticket delivery until one hour before the movie to reduce scalping.

3.2.2.4 Outputs
- Digital ticket issued to users.
- Purchase confirmation receipt.

3.2.2.5 Error Handling
- Payment failures shall cancel transactions.
- Ticket limits shall block excessive purchases.

3.2.3 Theater Rental

3.2.3.1 Introduction
The system shall allow users to reserve an entire theater for private showings.

3.2.3.2 Inputs
- Selected movie
- Date and time
- Theater location

3.2.3.3 Processing
- Apply flat-fee pricing.
- Override standard ticket purchase limits.

3.2.3.4 Outputs
- Rental confirmation.
- Digital access passes for invited guests.

3.2.3.5 Error Handling
- Conflicting reservations shall be rejected.

3.2.4 Discounts and Promotions

- The system shall support discounted tickets for seniors, students, and military-affiliated users.
- The system shall provide promotional discounts on movies and snacks.

- Users shall earn reward points redeemable for future discounts.

3.2.5 Ticket Sharing

- Users shall be able to share purchased tickets with others via phone numbers or email.
- Shared tickets shall remain tied to the original transaction.

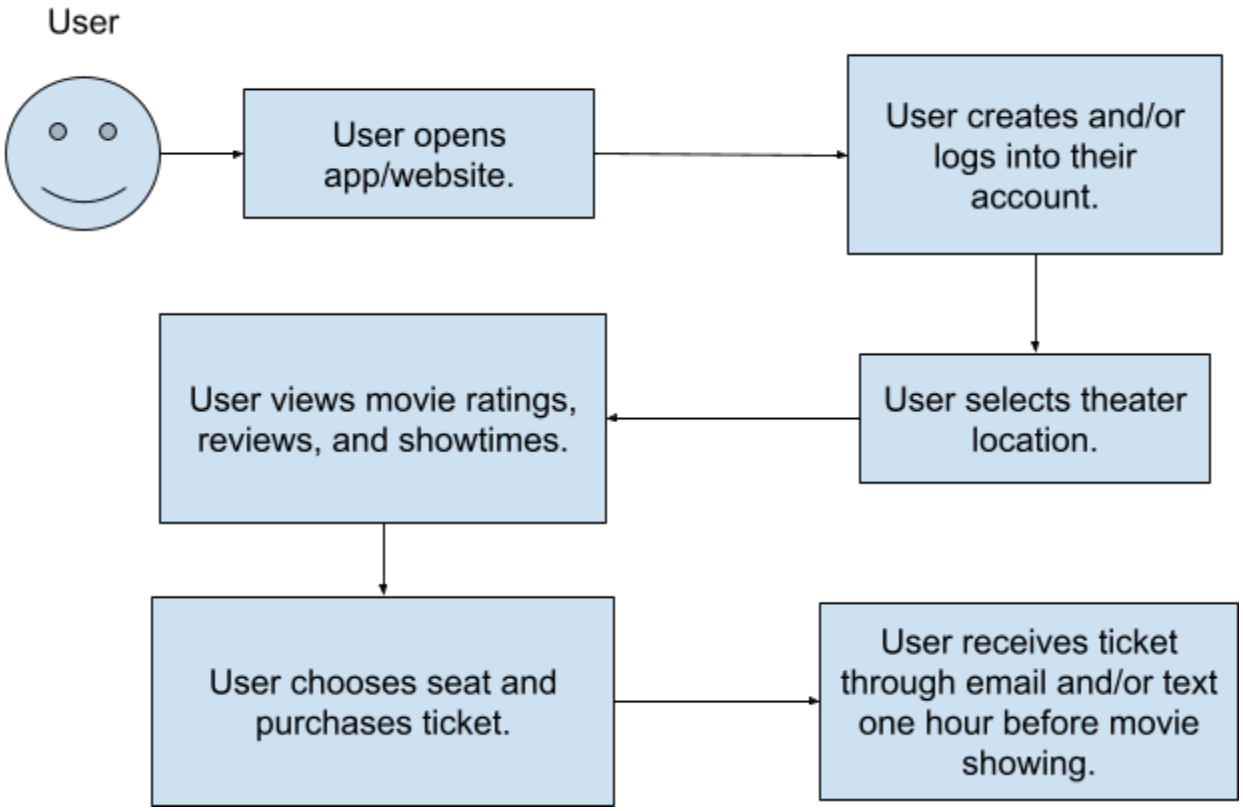
3.2.6 Cancellation and Refund Policy

- Users shall be allowed to cancel tickets up to 12 hours before showtime.
- Tickets purchased in large quantities may become non-refundable to prevent financial loss.

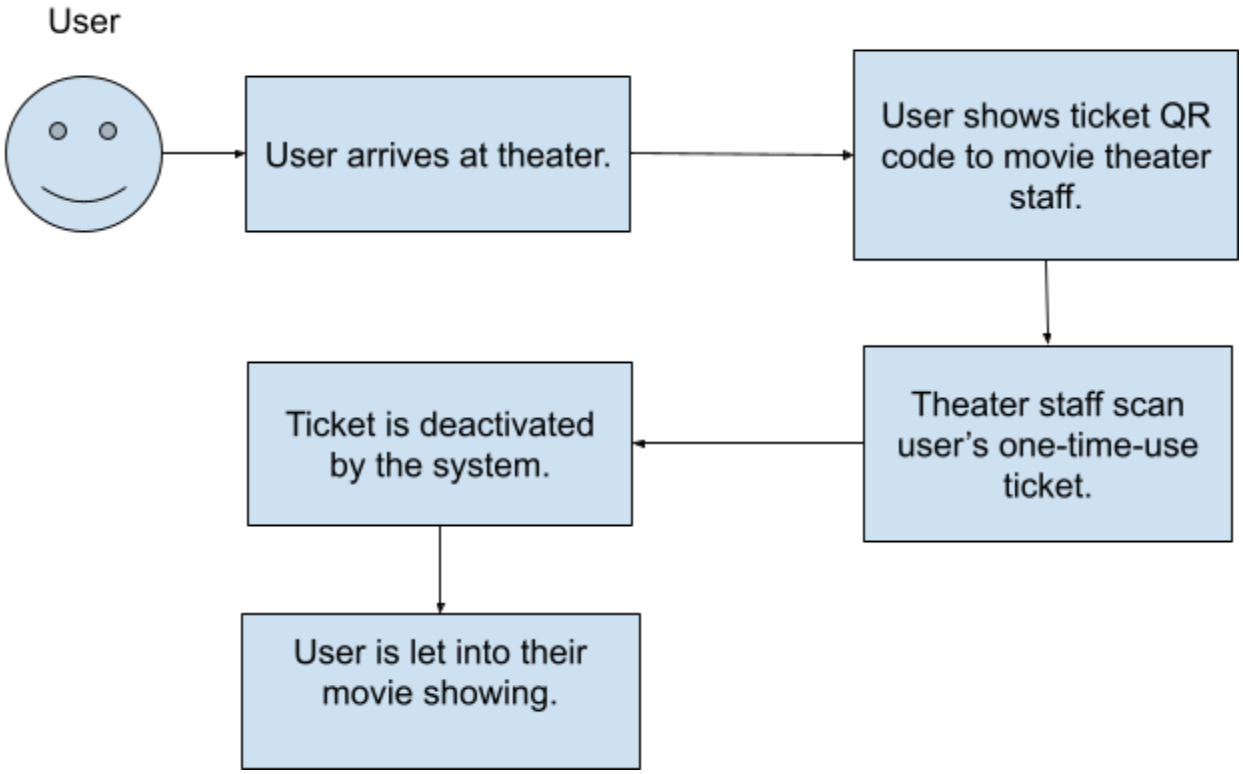
…

3.3 Use Cases

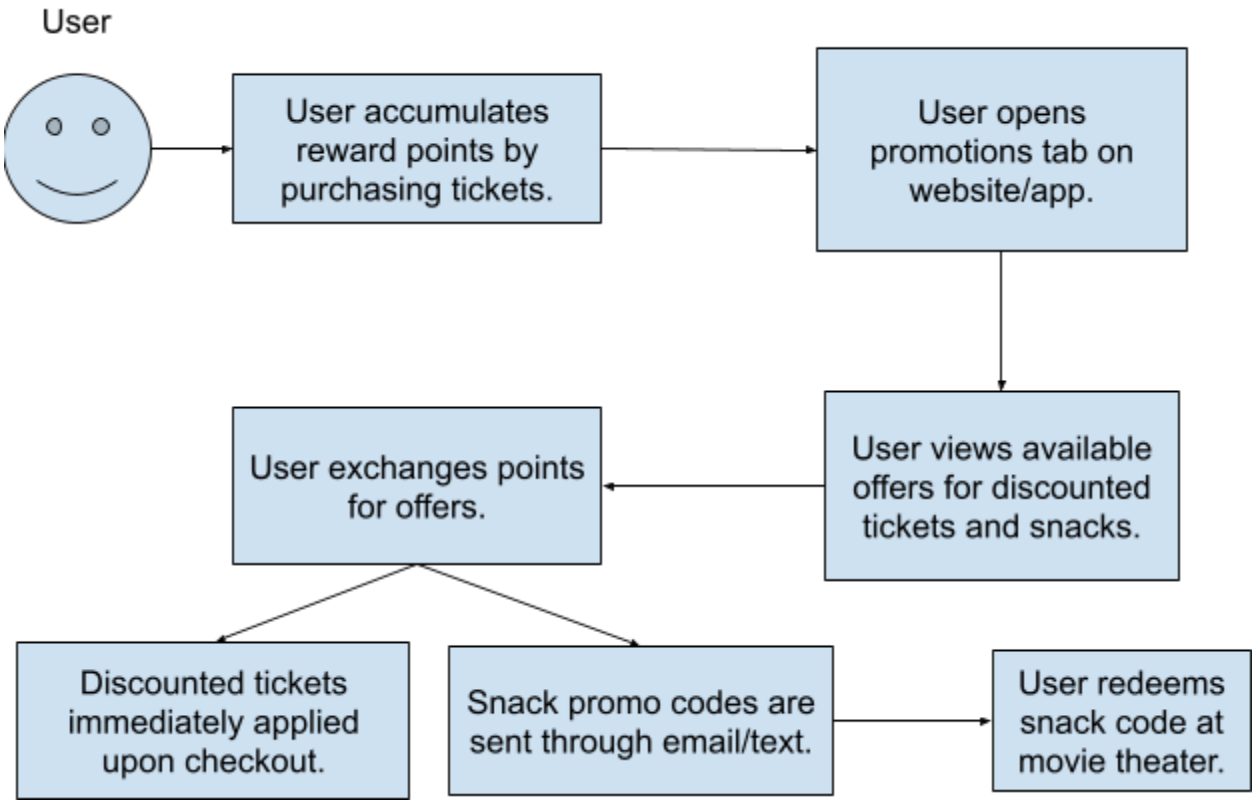
3.3.1 Use Case #1: User Purchases a Ticket



3.3.2 Use Case #2: User Enters the Theater



3.3.3 Use Case #3: User Collects Points



3.4 Classes / Objects

3.4.1 User

User
userID : long name : String email : String phoneNumber : long points : integer
createAccount() login() viewTickets()

3.4.2 Ticket

Ticket
ticketID : long showtime : long seatNumber : String isValid : boolean
generateTicket() : integer validateTicket() : boolean

3.5 Non-Functional Requirements

3.5.1 Performance

The system shall be designed to support a capacity of 10,000 concurrent users.

3.5.2 Reliability

The system will be required to have an uptime of at least 99%. Back-ups will take place regularly to preserve user data, including points and tickets. New and experimental features will be rigorously tested to ensure ease-of-use by users.

3.5.3 Availability

The software shall be available primarily for participating theaters in the United States. A web version will be available for all users, while the app version will be available on Android and iOS. The system shall be accessible at all times except during scheduled maintenance.

3.5.4 Security

The system shall provide two-factor authentication for users. User data shall be encrypted to protect all users' information in the unlikely event of a data breach. Payment data shall comply with PCI-DSS standards. Captchas will be put in place when entering the webpage or app for the first time for each IP address.

3.5.5 Maintainability

The system shall allow for weekly updates without full downtime. If necessary, updates will take place when traffic is expected to be low, such as after-hours of participating theaters.

3.5.6 Portability

The system shall function on all major operating systems via web browsers and mobile devices. The system shall have the same features between the web version and application version, though the user interface may be slightly different.

4. Analysis Models

4.1 Sequence Diagrams

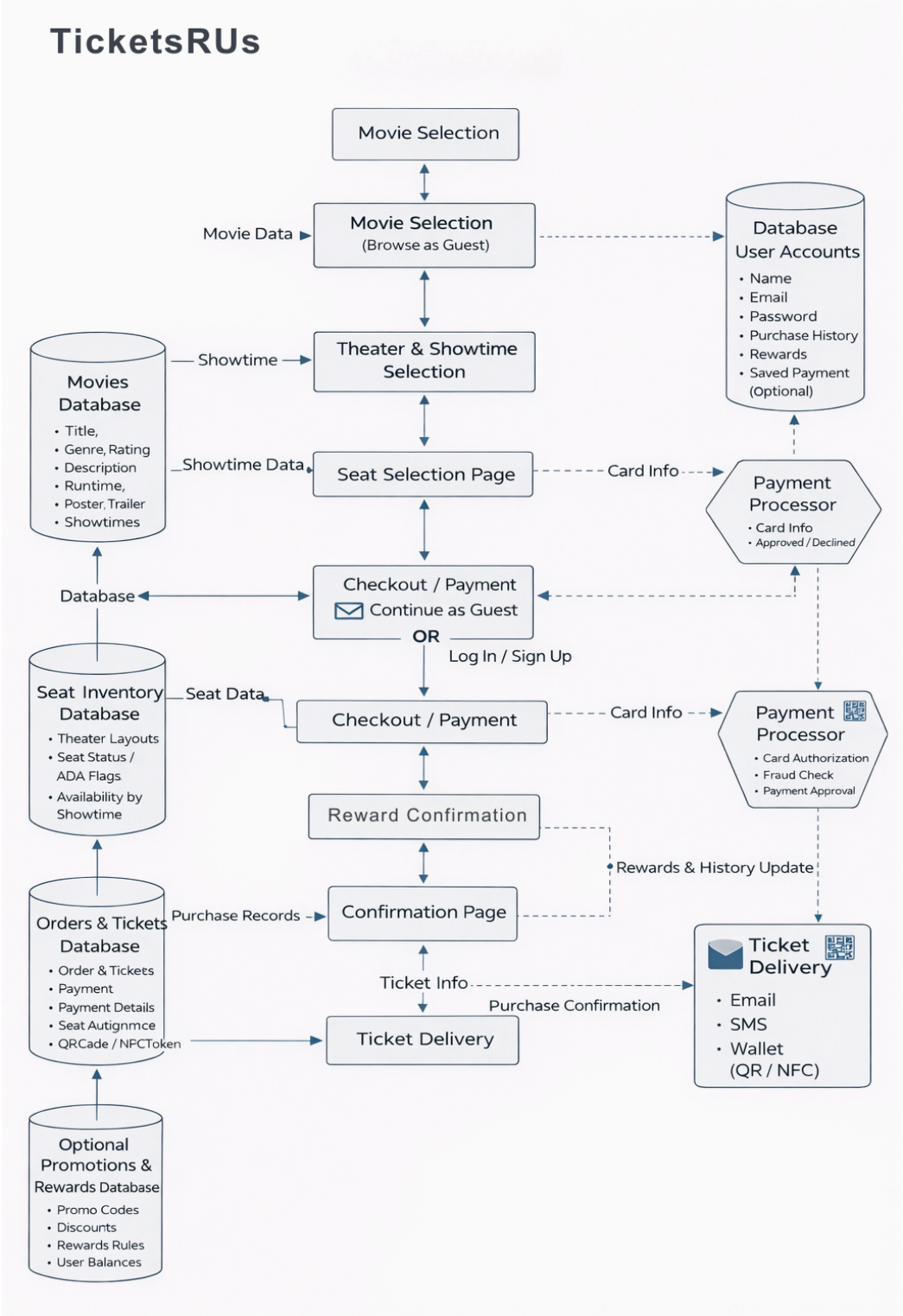


Figure 1: Software Architecture Diagram

4.2 Sequence Diagram Descriptions

4.2.1 Movie Selection

The system begins at the **Movie Selection** stage where all users can browse currently available movies. This stage allows users to explore available films without requiring authentication.

The **Movies Database** provides all relevant movie information, including:

- Movie titles
- Genres and ratings
- Movie descriptions
- Runtime information
- Movie poster and trailers
- Available showtimes

The user can browse the movie catalog freely as a guest before deciding to proceed with ticket selection.

4.2.2 Movie Selection (Browse as Guest)

At this stage, users interact with the platform as guest users, allowing them to explore available movie listings without logging in to an account.

This design improves accessibility and reduces barriers for new customers who wish to quickly view movie options before committing to a purchase.

The system retrieves movie data from the Movies Database and displays the available options to the user. If the user later decides to purchase tickets, they will be prompted to either:

- Continue as a guest
- Log in to an existing account
- Create a new account

4.2.3 Theater and Showtime Selection

After selecting a movie, users proceed to the **Theater and Showtime Selection** stage.

At this stage, the system displays:

- Available theaters showing the selected movie.

- Showtime schedules
- Theater location

The system retrieves this information from the **Movie Database**, which stores showtime data associated with each film.

Users select their preferred theater and showtime before proceeding to seat selection.

4.2.4 Seat Selection Page

Once a showtime is selected, the user enters the **Seat Selection Page** stage, where available seats for the chosen screening are displayed visually.

The system retrieves seat availability data from the **Seat Inventory Database**, which contains:

- Theater seating layouts
- Seat availability status
- ADA-accessible seating indicators
- Seat availability for each showtime

Users select their preferred seats, and the system temporarily reserves them during the checkout process to prevent double booking.

4.2.5 Checkout / Payment (Guest or Registered User)

After selecting seats, the user proceeds to the **Checkout / Payment** stage.

At this point, the system prompts the user with two options:

Continue as Guest

Users may proceed without creating an account. However, they must provide a valid email address to ensure that the ticket delivery can occur successfully.

Log in / Sign Up

Users may choose to log in or create a new account. Registered accounts allow the system to store additional information such as:

- Purchase history
- Reward points
- Saved Payment methods (optional)

User account information is stored and retrieved from the **User Account Database.**

4.2.6 Payment Processing

Once checkout details are confirmed, the system securely processes payment using an external payment processor.

The payment processor performs several critical tasks, including:

- Card authorization
- Fraud detection checks
- Transaction approval or rejection

Only secure card information required for the transaction is transmitted to the payment processor. TicketsRUs does not permanently store full card details, ensuring compliance with payment security standards.

If the payment is approved, the purchase proceeds to order confirmation.

4.2.7 Reward Confirmation

If the user is logged into an account, the system performs a **Reward Confirmation** step after payment approval.

During the process:

- Reward points may be earned or redeemed.
- The user's purchase history is updated.
- Loyalty benefits are applied if applicable.

Reward and promotion information is managed through the **Promotions and Rewards Database**, which stores:

- Promotional codes
- Discount rules
- Reward point balances
- Loyalty program deals/configurations

This step ensures that eligible users receive applicable benefits associated with their purchase.

4.2.8 Confirmation Page

After successful payment and reward processing, the system displays the **Confirmation Page**.

This page provides a summary of the completed transaction, including:

- Movie title
- Theater location
- Showtime details
- Selected seat numbers
- Total purchase amount
- Confirmation number

The system records transaction details in the **Orders and Tickets Database**, which stores:

- Order and ticket records
- Payment details
- Seat assignments
- Generated QR code and NFC ticket token

This database serves as the central record for all the tickets purchased.

4.2.9 Ticket Delivery

The final step in the process is **Ticket Delivery**.

The system retrieves ticket information from the **Orders and Tickets Database** and delivers the purchased tickets to the user through multiple supported methods:

- Email delivery
- SMS messaging
- Digital wallet integration using QR or NFC technology

The delivered tickets contain a scannable digital code used for entry validation at the theater. This process ensures that users receive their tickets after purchase, allowing them to present the digital ticket upon arrival at the theater.

4.2.10 Database Summary

The TicketsRUS system relies on several databases to support its operations:

4.2.10.1 Movies Database

Stores all movie-related information, including showtimes and media details.

4.2.10.2 Seat Inventory Database

Manages theater seating layouts and real-time seat availability.

4.2.10.3 User Account Database

Stores registered user information, including login credentials and purchase history.

4.2.10.4 Orders and Tickets Database

Maintains records of completed purchases, seat assignments, and digital ticket identifiers.

4.2.10.5 Promotions and Rewards Database

Handles promotional codes, discounts, reward points, and loyalty program data.

4.2.11 System Integration

The TicketsRUs platform integrates with external services to support payment processing and ticket delivery. These integrations ensure secure financial transactions and reliable communication with the customers while maintaining overall system scalability and performance.

All integrations within the TicketsRUs platform are designed to follow secure communication practices. Sensitive data such as payment information and personal account details are transmitted using encrypted connections to protect user privacy and prevent unauthorized access.

External services used by the system must comply with industry security standards to ensure that financial transactions and customer data are handled responsibly.

4.3 UML Diagram

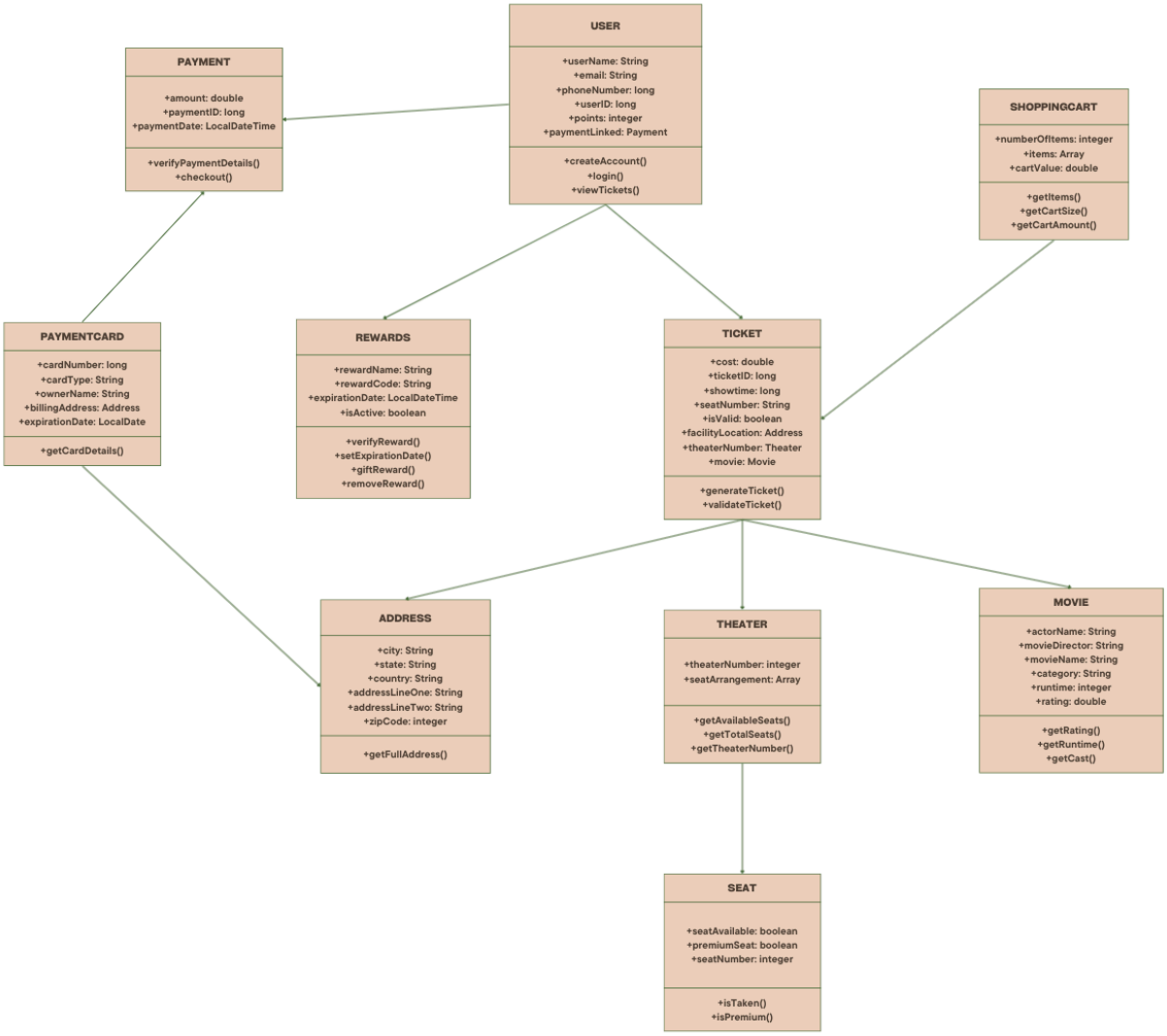


Figure 2: UML Diagram

4.4 UML Diagram Descriptions

4.4.1 User Class

Overview: This class maintains data directly related to the user of the software such as personal information.

Attributes: Both Strings, userName stores the user's name they selected upon account creation, and email simply stores their email. phoneNumber stores their cell number, and userID stores the user's identification number that is used to differentiate between users; both phoneNumber and userID are stored as longs. The points attribute stores the user's reward point balance.

paymentLinked stores the payment information the user has linked their account to, which is used when making purchases. Since the system only allows digital payments, this will always be a PaymentCard object (debit/credit card).

Operations: createAccount() allows the user to create their account using a username, email, and password. login() lets the user enter the website/and or application through their account information, and viewTickets() returns the history of the user's purchased tickets - both expired and non-expired ones.

Uses:
- Ticket class: Links tickets to the user
- Rewards class: Links rewards to the user

Used by: NONE

4.4.2 Ticket Class

Overview: The Ticket class maintains data related to the online movie tickets sold by our software.

Attributes: cost stores the ticket's price in local currency as a double, ticketID stores the ticket's unique identification number, and showtime stores the showing time of the movie listed on the ticket - both ticketID and showtime are stored as longs. seatNumber stores the seat location as a String, isValid is a boolean that stores false if the ticket is expired or has been used already (true otherwise), and facilityLocation uses the Address data type to store the location of the theater's building.

Operations: generateTicket() creates a ticket with all relevant information, including the ticketID, showtime, seatNumber, theaterLocation, theaterNumber, movieName, and information used to identify the customer (entered by customer during checkout, separate from User Class). validateTicket() returns isValid, which depends on if the ticket has expired or been used.

Uses:
- Address class: Stores facility address on ticket
- Movie class: Stores movie information on ticket
- Theater class: Stores theater information on ticket

Used by:
- User class
- ShoppingCart class

4.4.3 Rewards Class

Overview: The Rewards class stores data regarding rewards that the user can earn and redeem.

Attributes: rewardName stores the name of the reward, which will be displayed on the UI for the user. rewardCode is a unique String identifier made up of letters (A-Z, a-z) and numbers (0-9) that the user inputs to redeem their reward. expirationDate is a LocalDateTime object that stores the exact date and time the code expires. isActive is a boolean variable that represents whether the code can still be redeemed or not. It will be set to false if the code is already redeemed or if the expiration date passes.

Operations: verifyReward() checks the rewardCode and isActive to determine whether the reward is valid and can be redeemed. setExpirationDate() sets the expirationDate attribute to a specific future date that will determine when the code is no longer valid. giftReward() allows the user to see the reward in their user interface, while removeReward() removes it from their viewing and sets the isActive boolean variable to false.

Uses: NONE

Used by:
- User class

4.4.4 Address Class

Overview: The Address class stores information regarding physical and billing addresses, including the zip code, city, etc.

Attributes: city is a String that stores the listed city on the address. state is a String that stores the state the address is in. country is a String that stores the country the address is in. addressLineOne and addressLineTwo are Strings that store more specific information about the address, like building number and streetName. addressLineOne needs to be filled while addressLineTwo can be left blank. zipCode is a five digit integer that represents the zip code listed on the address.

Operations: getFullAddress() returns a formatted string formed in a way that an address is typically written. It concatenates all listed attributes above.

Uses: NONE

Used by:
- PaymentCard class
- Ticket class

4.4.5 Movie Class

Overview: The Movie class stores details related to the movie.

Attributes: actorName stores the names of actors in the movie individually, movieDirector stores the name of the director, movieName stores the name of the movie, and category stores the classification of the movie; all of these attributes are Strings. runtime stores the movie length as an integer, and rating stores the movie's rating as a double.

Operations: getRating() returns the rating of a specific movie as a double, getRuntime() returns the length of the movie as an integer, and getCast() returns the names of actors of the movie as Strings.

Uses: NONE

Used by:
- Ticket class

4.4.6 ShoppingCart Class

Overview: The ShoppingCart class holds all information regarding the user's items they wish to purchase.

Attributes: numberOfItems is an integer that stores the total number of objects currently in the user's shopping cart. items is an array that stores the actual objects listed in the shopping cart. These will be Ticket objects. cartValue is a double that stores the current price of all the items in the user's shopping cart.

Operations: getItems() iterates through all the Ticket objects in the items array. getCartSize() counts the number of objects in the items array and stores it in the numberOfItems attribute. getCartAmount() iterates through all objects in the items array, gets each of their prices, and adds them to the cartValue attribute.

Uses:
- Ticket class: Stores Ticket objects in the array of items

Used by: NONE

4.4.7 PaymentCard Class

Overview: The PaymentCard class maintains data regarding the card the user is paying with and inherits the Payment class.

Attributes: cardNumber stores the card's number as a long, cardType stores the type of card as a String (debit or credit), and ownerName stores the name of the cardholder as a String. billingAddress uses the Address datatype to store the card's billing information, and expirationDate uses Java's built-in LocalDate object to store the card's expiry.

Operations: getCardDetails() returns all card information: the cardNumber, cardType, ownerName, billingAddress, and expirationDate.

Uses:
- Address class: Stores billing information in billingAddress using the Address datatype

Used by:
- Ticket class

4.4.8 Theater Class

Overview: The Theater class represents the individual theaters within a participating location. It stores all the seats and the arrangement of those seats.

Attributes: theaterNumber is an integer that represents the number of the theater at the participating location. seatArrangement is a 2D array that represents the layout of the theater. Each subsequent array is the letter zone (A, B, C, etc.) while the individual seats hold their designated number.

Operations: getAvailableSeats() returns the total number of available seats in the theater. getTotalSeats() returns the total number of seats in the theater, regardless of if they are reserved. getTheaterNumber() returns the number of the theater as listed in the theaterNumber parameter.

Uses:
- Seat class: Stores seat information in the seatArrangement array using the Seat datatype

Used by:
- Ticket class

4.4.9 Seat Class

Overview: The Seat class stores information about theater seating.

Attributes: seatAvailable is a boolean that is true if a seat is open and false if not. premiumSeat is a boolean that is true if the seat is a premium seat and false if not. seatNumber stores the seat's location number as an integer.

Operations: isTaken() returns seatAvailable, and isPremium() returns premiumSeat.

Uses: NONE

Used by:
- Theater class

4.4.10 Payment Class

Overview: The Payment class stores information relating to the transaction the user makes.

Attributes: amount is a double that represents the total charge of the current payment. paymentID is a long that represents a unique identifier for the specific payment. It allows the system administrators to reference a specific payment number if needed. paymentDate is a LocalDateTime object that stores the specific date and time the transaction was made.

Operations: verifyPaymentDetails() verifies the details of the listed payment type with banking networks to ensure that all the listed information, like billing address and card number, is valid, while checkout() completes the transaction between the user and the software.

Uses: NONE

Used by:
- User class

4.5 Development Plan and Timeline

4.5.1 Partitioning of Tasks

a. **Hincklair Patino** - System architecture, documentation, overall project structure
    i. Drafting and planning of project structure and architecture, while documenting each process.
b. **Sean Lumahan** - Coding, technical implementation of the system
    i. Developing the core application logic, handling database interactions, implementing the checkout and payment workflow, and ensuring system functions.
c. **Brady Santos** - Visual design, user interface, and graphical components of the project
    i. Assisting with diagrams, system visuals, interface concepts, and any additional graphics necessary for documentation and presentation.

4.5.2 Expected Development Timeline

There will be three stages of development for TicketsRUs:

a. System planning and architectural design
    i. Expected time for completion: 2 months
b. Implementation of system functionalities
    i. Expected time for completion: 4 months
c. Testing, refinement, and presentation preparation
    i. Expected time for completion: 3 months
    ii. More time may be allocated if testing finds a significant need for refinement

5. System Test Plan

5.1 System Test Cases

Test Case Link: [GitHub Test Cases]

TicketsRUs Test Cases.xlsx

TicketsRUs Test Cases

TestCaseId	Test Type	Component	Priority	Description/Test Summary	Pre-requisites	Test Steps	Expected Result	Actual Result	Status	Test Executed By
LoginEmail_1	Unit	Login_Email_Field	P0	Verify that the login email field accepts a properly formatted email address.	Login page is displayed.	1. Open the Login page. 2. Enter user@example.com in the email field. 3. Click outside the field.	The field accepts the input and no validation error is shown.	The email field accepted the properly formatted email address and did not display an error.	Pass	Brady Santos
LoginEmail_2	Unit	Login_Email_Field	P0	Verify that the login email field rejects an invalid email format.	Login page is displayed.	1. Open the Login page. 2. Enter userexample.com in the email field. 3. Attempt to continue.	The system displays an email format validation error and does not proceed.	The email field did not reject the invalid email input and allowed the user to proceed.	Fail	Brady Santos
LoginPassword_1	Unit	Password_Field	P0	Verify that the password field masks characters as the user types.	Login page is displayed.	1. Open the Login page. 2. Enter Ticket123! in the password field. 3. Observe the field display.	The password is masked and not shown in plain text.	The password was masked as the user entered characters.	Pass	Brady Santos
GuestEmail_1	Unit	Guest_Email_Confirmation	P0	Verify that guest checkout requires matching email and confirm email values.	Seats have been selected and the guest checkout prompt is displayed.	1. Choose Continue as Guest. 2. Enter an email address. 3. Enter a different value in Confirm Email. 4. Attempt to continue.	The system blocks checkout and displays an email mismatch message.	The system did not allow the guest to proceed and displayed a mismatch message.	Pass	Brady Santos
SeatHold_1	Unit	Seat_Hold_Logic	P1	Verify that a selected seat is temporarily held during checkout.	A valid movie, showtime, and seat are available.	1. Select an available seat. 2. Proceed to Checkout / Payment. 3. Check seat availability status for the selected seat.	The selected seat is marked as temporarily held and is not shown as available to other users.	The selected seat was marked as held and was temporarily unavailable to other users.	Pass	Brady Santos
MovieBrowse_1	Functional	Movie_Selection	P0	Verify that users can browse available movies without logging in.	Movies Database contains active movie listings.	1. Open the TicketsRUs home page. 2. Observe the movie list before logging in.	Available movies are displayed without requiring the user to log in.	The user was asked to log in before being shown available movies.	Fail	Brady Santos
ShowtimeSelect_1	Functional	Theater_Showtime_Selection	P0	Verify that selecting a movie displays only theaters and showtimes for that movie.	At least one movie has valid showtimes in the database.	1. Select a movie from Movie Selection. 2. Open Theater & Showtime Selection.	The system displays only theaters and showtimes associated with the selected movie.	The system displayed the selected movie's theaters and showtimes.	Pass	Brady Santos
SeatSelect_1	Functional	Seat_Selection	P0	Verify that the seat map shows available, unavailable, and ADA seats correctly.	A showtime with a configured theater layout exists.	1. Select a movie and showtime. 2. Open Seat Selection Page. 3. Review seat labels and availability states.	The seat map correctly displays seat availability and ADA-designated seats.	The seat map properly displayed the seats' availabilities and labeled ADA seats.	Pass	Sean Lumahan
GuestCheckout_1	Functional	Checkout_Payment	P0	Verify that a guest user can proceed to checkout after entering a valid email.	Seats have been selected and guest checkout is available.	1. Choose Continue as Guest. 2. Enter matching email and confirm email values. 3. Continue to payment.	The system allows the guest user to continue to payment.	The system allowed the guest to continue to payment.	Pass	Sean Lumahan
RewardApply_1	Functional	Promotions_Rewards	P1	Verify that eligible reward points or promotions are applied during checkout for a logged-in user.	User is logged in and has an active reward balance or valid promo code.	1. Log in to an eligible account. 2. Select movie, showtime, and seats. 3. Apply rewards or a valid promo code during checkout.	The discount is applied correctly and the updated total is shown before payment.	The discount was calculated incorrectly, but the updated total was shown.	Fail	Sean Lumahan
LoginPurchase_1	System	Logged-In_Purchase	P0	Verify that a registered user can log in, purchase tickets, and receive ticket delivery successfully.	A valid user account exists and a movie/showtime/seat is available.	1. Log in to TicketsRUs. 2. Browse movies and select a showtime. 3. Select seats. 4. Complete payment.	The order is completed, purchase history is updated, and the digital ticket is delivered successfully.	The order was completed, purchase history was updated, and the digital ticket	Pass	Sean Lumahan
GuestPurchase_1	System	Guest_Purchase	P0	Verify that a guest user can complete a ticket purchase and receive the ticket by email or SMS.	A movie, showtime, and seat are available and the payment processor is online.	1. Browse as guest. 2. Select movie, showtime, and seats. 3. Choose Continue as Guest. 4. Enter matching email values.	The guest purchase is completed successfully and the ticket is delivered to the provided contact method.	The guest successfully purchased the ticket and received it through SMS after selecting SMS as	Pass	Sean Lumahan
PaymentDeclineE_1	System	Payment_Failure_Handling	P0	Verify that the system handles a declined payment without confirming the order.	A movie, showtime, and seat are available and the payment processor can return a decline response.	1. Proceed to Checkout / Payment. 2. Enter payment details that trigger a decline. 3. Submit payment.	The system displays a payment failure message, does not create a confirmed order, and does not deliver a ticket.	The system displayed a payment failure message and neither created a confirmed	Pass	Sean Lumahan
TicketDelivery_1	System	Ticket_Delivery	P1	Verify that a successful purchase generates a valid digital ticket for delivery.	A completed paid order exists in the Orders & Tickets Database.	1. Complete a valid purchase. 2. Open the delivered ticket from email, SMS, or wallet. 3. Review ticket details and scannable code.	The delivered ticket contains the correct movie, showtime, seat details, and a valid QR or NFC token.	The delivered ticket contained the correct movie showing details and had a valid	Pass	Sean Lumahan
SeatConflict_1	System	Concurrent_Seat_Reservation	P0	Verify that two users cannot successfully purchase the same seat for the same showtime.	Two active sessions are available and the same seat is open for one showtime.	1. User A selects a seat and proceeds to checkout. 2. User B attempts to select the same seat for the same showtime.	Only one transaction succeeds for the seat; the other user receives a seat unavailable message.	The transaction succeeded for both User A and User B. A seat unavailable message	Fail	Sean Lumahan